Лабораторна робота №1

Кореляційний аналіз у задачах виявлення прихованих об’єктів

Мета: Вивчити теоретичні основи та обмеження класичного кореляційного аналізу. Опанувати математичний апарат та фізичний зміст функції взаємної кореляції. Набути прак тичних навичок застосування ковзного кореляційного аналізу для виділення детермінованого корисного сигналу на тлі інтенсивного випадкового шуму

Варіант №3

Завдання 1. Розробити клієнтський додаток для взаємодії із сервером ехолота.

```
import requests
import matplotlib.pyplot as plt
BASE_URL = "https://ooi-lab1.up.railway.app"
SIGNAL_PATH = "/api/sensor?step={step}"
session = requests.Session()

def get_json(path):
    response = session.get(BASE_URL + path, timeout=10)
    response.raise_for_status()
    return response.json()
```

Завдання 2. Отримати з сервера поточні параметри конфігурації та еталонний імпульс.

```
cfg = get_json("/api/config")
template = get_json("/api/pulse")
```

Завдання 3. Запитувати зашумлений сигнал для кожної точки на заданому маршруті зондування.

```
SIGNAL_PATH = "/api/sensor?step={step}"

def get_signal(step):
    data = get_json(SIGNAL_PATH.format(step=step))
    result = extract_list(data)
    if result is None:
        raise ValueError("У відповіді немає масиву: " + str(data)[:200])
    return result
```

Завдання 4. Самостійно написати код для обчислення функції взаємної кореляції між еталонним імпульсом та отриманим сигналом.

```
def cross_correlation(signal, template):
    n = len(signal)
    m = len(template)
    result = []
    for k in range(n - m + 1):
        total = 0.0
        for j in range(m):
            total += signal[k + j] * template[j]
        result.append(total)
    return result
```

Завдання 5. Знайти індекс зсуву, котрий відповідає максимальному значенню кореляційної функції.

```
def find_peak(correlation):
    best_k = 0
    best_value = correlation[0]
    for k in range(1, len(correlation)):
        if correlation[k] > best_value:
            best_value = correlation[k]
            best_k = k
    return best_k
```

Завдання 6. Перетворити знайдений часовий зсув на фізичне значення глибини.

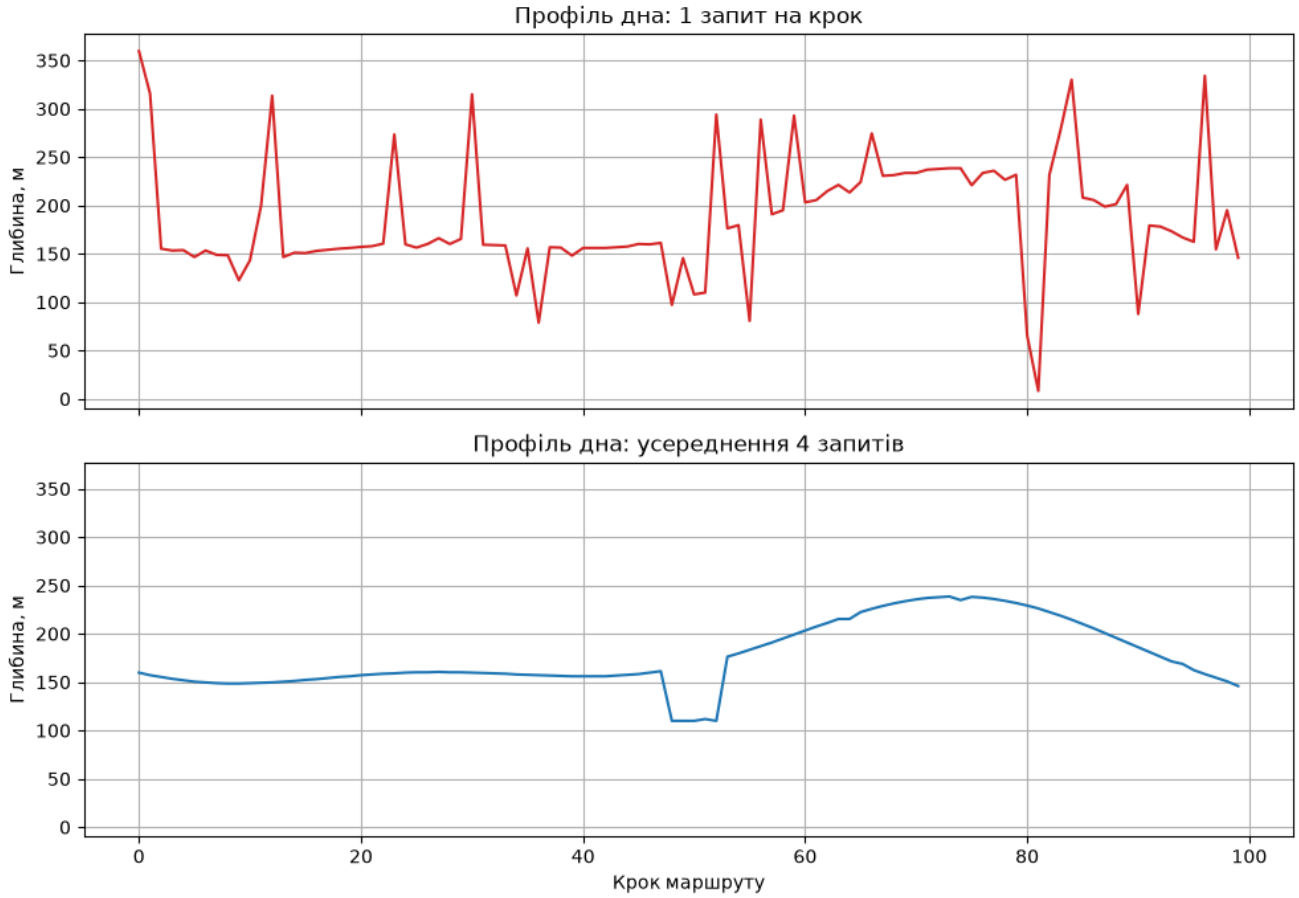
```
def shift_to_depth(k, sample_rate, sound_speed):
    delay = k / sample_rate
    return delay * sound_speed / 2
```

Завдання 7. Виконати цей алгоритм для всіх кроків маршруту та побудувати візуальний графік профілю дна.

```
def build_profile(template, cfg, repeats):
    profile = []
    total = cfg["total_steps"]
    for step in range(total):
        profile.append(measure_depth(step, template, cfg, repeats))
        if step % 10 == 0:
            print(f"  крок {step}/{total}")
    return profile
```

Завдання 8. Розрахувати максимальну глибину вимірювання, котру дозволяють технічні параметри системи.

```
n_samples = len(get_signal(0))
theory_max = shift_to_depth(n_samples - len(template),
                            cfg["sample_rate"], cfg["sound_speed"])
```



Завдання 9. Перевірити покращення якості розрахунку глибини завдяки усередненню 3–4 запитів на кожному кроці, аби придушити хибні піки від шуму. Цей ефект слід продемонструвати наочно, порівнявши профіль дна від одного запиту з результатом, де дані накопичувалися та усереднювалися перед обчисленням кореляції.

Повний лістинг:

```
import requests
import matplotlib.pyplot as plt

BASE_URL = "https://ooi-lab1.up.railway.app"
SIGNAL_PATH = "/api/sensor?step={step}"
AVERAGE_COUNT = 4
OUTLIER_THRESHOLD = 3.0
session = requests.Session()
def get_json(path):
    response = session.get(BASE_URL + path, timeout=10)
    response.raise_for_status()
    return response.json()
def extract_list(data):
    if isinstance(data, list):
        return data
    if isinstance(data, dict):
        for value in data.values():
            if isinstance(value, list):
                return value
    return None
```

```
def get_signal(step):
    data = get_json(SIGNAL_PATH.format(step=step))
    result = extract_list(data)
    if result is None:
        raise ValueError("У відповіді немає масиву: " + str(data)[:200])
    return result
def cross_correlation(signal, template):
    n = len(signal)
    m = len(template)
    result = []
    for k in range(n - m + 1):
        total = 0.0
        for j in range(m):
            total += signal[k + j] * template[j]
        result.append(total)
    return result
def find_peak(correlation):
    best_k = 0
    best_value = correlation[0]
    for k in range(1, len(correlation)):
if correlation[k] > best_value:
            best_value = correlation[k]
            best_k = k
    return best_k
def shift_to_depth(k, sample_rate, sound_speed):
    delay = k / sample_rate
    return delay * sound_speed / 2
def average_signals(signals):
    count = len(signals)
    length = len(signals[0])
    averaged = []
    for i in range(length):
        total = 0.0
        for s in signals:
            total += s[i]
        averaged.append(total / count)
    return averaged
def measure_depth(step, template, cfg, repeats):
    signals = [get_signal(step) for _ in range(repeats)]
    signal = average_signals(signals) if repeats > 1 else signals[0]
    correlation = cross_correlation(signal, template)
    k = find_peak(correlation)
    depth = shift_to_depth(k, cfg["sample_rate"], cfg["sound_speed"])
    return depth, k
def build_profile(template, cfg, repeats, verbose=False):
    profile = []
    total = cfg["total_steps"]
    for step in range(total):
        depth, k = measure_depth(step, template, cfg, repeats)
        profile.append(depth)
        if verbose:
            print(f"Крок {step}: Зсув = {k}, Глибина = {depth:.2f} м")
        else:
            if step % 10 == 0 or step == total - 1:
print(f"\r  [~] Обробка: пройдено {step + 1}/{total} точок...", end="", flush=True)
    if not verbose:
        print("\r  [+] Сканування успішно завершено!                    ")
    return profile
def roughness(profile):
    total = 0.0
    for i in range(1, len(profile)):
        total += abs(profile[i] - profile[i - 1])
    return total / (len(profile) - 1)
def count_outliers(profile):
    count = 0
```

```
        for i in range(1, len(profile) - 1):
            left = abs(profile[i] - profile[i - 1])
            right = abs(profile[i] - profile[i + 1])
            if left > OUTLIER_THRESHOLD and right > OUTLIER_THRESHOLD:
                count += 1
    return count

cfg = get_json("/api/config")
template = get_json("/api/pulse")
print("Конфіг:", cfg)
print("Довжина імпульсу:", len(template), "відліків")
print("\nОдин запит на крок:")
profile_single = build_profile(template, cfg, 1, verbose=False)
print(f"\nУсереднення {AVERAGE_COUNT} запитів на крок:")
profile_avg = build_profile(template, cfg, AVERAGE_COUNT, verbose=True)
max_single = max(profile_single)
max_avg = max(profile_avg)
n_samples = len(get_signal(0))
theory_max = shift_to_depth(n_samples - len(template),
                            cfg["sample_rate"], cfg["sound_speed"])

print(f"Макс. глибина (1 запит):  {max_single:.2f} м")
print(f"Макс. глибина (усеред.):  {max_avg:.2f} м")
print(f"Теоретична межа системи:  {theory_max:.2f} м")
print(f"Шорсткість:  1 запит = {roughness(profile_single):.3f},  усереднення =
{roughness(profile_avg):.3f}")
print(f"Викидів:     1 запит = {count_outliers(profile_single)},  усереднення =
{count_outliers(profile_avg)}")

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True, sharey=True)
axes[0].plot(profile_single, color="tab:red")
axes[0].set_title("Профіль дна: 1 запит на крок")
axes[1].plot(profile_avg, color="tab:blue")
axes[1].set_title(f"Профіль дна: усереднення {AVERAGE_COUNT} запитів")
for ax in axes:
    ax.invert_yaxis()
    ax.set_ylabel("Глибина, м")
    ax.grid(True)
axes[1].set_xlabel("Крок маршруту")
plt.tight_layout()
plt.show()
```

```
E:\обчисл\lab1_bubon\.venv\Scripts\python.exe E:\обчисл\lab1_bubon\main.py
Конфіг: {'total_steps': 100, 'sample_rate': 2000, 'sound_speed': 1500.0, 'carrier_freq': 400.0}
Довжина імпульсу: 40 відліків

Один запит на крок:
  [+] Сканування успішно завершено!

Усереднення 4 запитів на крок:
Крок 0: Зсув = 426, Глибина = 159.75 м
Крок 1: Зсув = 419, Глибина = 157.12 м
Крок 2: Зсув = 414, Глибина = 155.25 м
Крок 3: Зсув = 409, Глибина = 153.38 м
Крок 4: Зсув = 405, Глибина = 151.88 м
Крок 5: Зсув = 401, Глибина = 150.38 м
Крок 6: Зсув = 399, Глибина = 149.62 м
Крок 7: Зсув = 397, Глибина = 148.88 м
Крок 8: Зсув = 396, Глибина = 148.50 м
Крок 9: Зсув = 396, Глибина = 148.50 м
```

```
Крок 91: Зсув = 483, Глибина = 181.12 м
Крок 92: Зсув = 470, Глибина = 176.25 м
Крок 93: Зсув = 457, Глибина = 171.38 м
Крок 94: Зсув = 450, Глибина = 168.75 м
Крок 95: Зсув = 433, Глибина = 162.38 м
Крок 96: Зсув = 422, Глибина = 158.25 м
Крок 97: Зсув = 412, Глибина = 154.50 м
Крок 98: Зсув = 402, Глибина = 150.75 м
Крок 99: Зсув = 389, Глибина = 145.88 м
Макс. глибина (1 запит):  359.62 м
Макс. глибина (усеред.):  238.50 м
Теоретична межа системи:  360.00 м
Шорсткість:  1 запит = 41.212,  усереднення = 3.193
Викидів:     1 запит = 51,  усереднення = 27
```

Рис. 1. Результат виконання.

Висновок: Згідно з результатами накопичення та усереднення даних перед обчисленням суттєво підвищує точність. На верхньому графіку (стр 2), котрий відображає профіль дна за одним запитом, спостерігається значна кількість різких стрибків глибини. Ці аномалії виникають, оскільки алгоритм неправильно ідентифікує випадковий шум замість ехо-сигналу.

Натомість нижній графік демонструє профіль дна після усереднення 4 запитів на кожному кроці. Ця крива є згладженою, себто система применшає вплив перешкод, аби виявити правильні дані.

Розраховані метрики також підтверджують ефективність методу: показник шорсткості профілю впав із 41.212 до 3.193, а кількість критичних викидів зменшилася з 51 до 27. Отже, використання кореляційного аналізу разом із попереднім усередненням зашумленого сигналу дозволяє надійно виділяти прихований об'єкт на тлі інтенсивного випадкового шуму.